Chapter 6: Methods

COMP 110
Java examples, discussion, and practice

Learning goals

Define and call methods with parameters
Choose a return type and use returned values
Trace pass by value and variable scope
Resolve overloaded method calls
Design a program with small, reusable methods

Opening problem: repeated sums

Find the sums from 1 through 10,
20 through 30, and 35 through 45.

Discussion: What stays the same?
What changes between the three calculations?

One calculation, repeated code

```
int sum = 0;
for (int i = 1; i <= 10; i++) {
    sum += i;
}
System.out.println(sum);
// Repeat the loop for 20..30 and 35..45
```

Changing a repeated loop means updating several places.

A reusable range-sum method

```
public static int sumRange(int start, int end) {
    int sum = 0;
    for (int i = start; i <= end; i++) {
        sum += i;
    }
    return sum;
}
```

The endpoints become parameters. The result leaves through return.

Calling sumRange

```
System.out.println(sumRange(1, 10));
System.out.println(sumRange(20, 30));
System.out.println(sumRange(35, 45));

// Output
55
275
440
```

A call supplies arguments for one execution of the method.

Parts of a method

```
public static int max(int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}
```

Header: modifiers, return type, name, parameters. Body: statements.

Parameters and arguments

Definition: max(int a, int b)
Parameters a and b receive values.

Call: max(7, 12)
Arguments 7 and 12 supply those values.

Position matters: first goes to a, second to b.

Method signature

For these methods, the signature consists of the name and ordered parameter types.

max(int, int) differs from max(double, double).

Parameter names and return type do not distinguish overloaded methods.

A returned value continues the expression

```
int larger = max(7, 12);
int total = 3 + max(4, 9);
System.out.println(larger);
System.out.println(total);
```

Trace the method call before finishing the assignment.

Quiz 1: method vocabulary

1. In max(7, 12), which items are arguments?

2. What is the signature of
   static int square(int number)?

3. Does changing a parameter name create
   a new overload?

A value must return on every normal path

```
public static int sign(int n) {
    if (n > 0) return 1;
    if (n == 0) return 0;
    return -1;
}
```

The final return covers the remaining case.

void methods perform an action

```
public static void nPrintln(String message, int n) {
    for (int i = 0; i < n; i++) {
        System.out.println(message);
    }
}

nPrintln("Welcome to Java", 3);
```

This call prints the message on three separate lines.

Discussion: return a grade or print it?

A grade calculator can return a char:
char grade = letterGrade(85);

A reporting method can print that char:
System.out.println(grade);

Which design works in a console program,
a web form, and an automated test?

A method that returns a letter grade

```
public static char letterGrade(int score) {
    if (score >= 90) return 'A';
    if (score >= 80) return 'B';
    if (score >= 70) return 'C';
    if (score >= 60) return 'D';
    return 'F';
}
```

Precondition: score is an integer from 0 through 100.

Reuse from another class

```
public class NumberTools {
    public static int square(int n) {
        return n * n;
    }
}

// Inside a method in another class:
int result = NumberTools.square(6);
```

ClassName.methodName calls an accessible static method.

Pass by value: a copied primitive

```
int x = 1;
increment(x);
System.out.println(x);

static void increment(int value) {
    value++;
    System.out.println(value);
}
```

Prediction: What prints inside the method and after the call?

Tracing the copied value

Before the call: caller x = 1

At entry: parameter value = 1

After value++: parameter value = 2

After return: caller x = 1

Why this swap leaves the caller unchanged

```
static void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int first = 5, second = 10;
swap(first, second);
System.out.println(first + " " + second);
```

The parameters swap their copies. Predict the caller output.

A return value can update the caller

```
static int incremented(int value) {
    return value + 1;
}

int x = 1;
x = incremented(x);
System.out.println(x);
```

The assignment changes x in the caller.

Quiz 2: tracing calls

Given int n = 4; and
static int twice(int x) { return x * 2; }

1. twice(n); then print n. What prints?
2. n = twice(n); then print n. What prints?
3. Can int y = nPrintln("Hi", 2); compile?

Modularizing a program

main coordinates input and output.

gcd(a, b) computes a mathematical result.
isPrime(n) answers a yes/no question.

Discussion: Which method could you test without using the keyboard?

Case study: greatest common divisor

```
static int gcd(int a, int b) {
    int k = Math.min(a, b);
    while (k >= 2) {
        if (a % k == 0 && b % k == 0) {
            return k;
        }
        k--;
    }
    return 1;
}
```

Precondition: a and b are positive integers.

Case study: a reusable prime test

```
static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int divisor = 2; divisor <= n / divisor;
         divisor++) {
        if (n % divisor == 0) return false;
    }
    return true;
}
```

Tests: 1 is false, 2 is true, 9 is false, 13 is true.

Using isPrime to find 50 primes

```
int count = 0;
int candidate = 2;
while (count < 50) {
    if (isPrime(candidate)) {
        count++;
        System.out.printf("%4d", candidate);
        if (count % 10 == 0) System.out.println();
    }
    candidate++;
}
```

The loop chooses candidates. isPrime decides whether to print.

Quiz 3: tests and contracts

1. What should gcd(9, 12) return?
2. Why does isPrime(1) return false?
3. Which grade tests check a threshold:
   84 and 85, or 89 and 90?
4. Which gcd inputs violate our precondition:
   (5, 7), (0, 7), or (-2, 6)?

Hexadecimal conversion

Hexadecimal digits: 0–9 and A–F
A = 10, B = 11, ..., F = 15

For "2AF", accumulate one digit at a time:
0 × 16 + 2 = 2
2 × 16 + 10 = 42
42 × 16 + 15 = 687

Converting one hexadecimal digit

```
static int hexDigitToDecimal(char ch) {
    if (ch >= '0' && ch <= '9') {
        return ch - '0';
    }
    return ch - 'A' + 10;
}
```

Precondition: ch is 0–9 or A–F.

Converting a hexadecimal string

```
static int hexToDecimal(String hex) {
    int value = 0;
    for (int i = 0; i < hex.length(); i++) {
        int digit = hexDigitToDecimal(hex.charAt(i));
        value = value * 16 + digit;
    }
    return value;
}
```

Practice: trace hexToDecimal("1B") on paper.

Overloading methods

Overloads share a name and have different parameter type lists.

max(int, int)
max(double, double)

The compiler selects an applicable overload using the arguments' compile-time types.

Overload selection examples

```
static int max(int a, int b) {
    return a > b ? a : b;
}
static double max(double a, double b) {
    return a > b ? a : b;
}

max(3, 5);
max(3.0, 5.0);
max(3, 5.0);
```

Which overload handles each call?

An ambiguous method call

```
static double max(int a, double b) {
    return a > b ? a : b;
}
static double max(double a, int b) {
    return a > b ? a : b;
}

max(1, 2);  // compile-time error
```

Both overloads apply. Neither is more specific overall.

Quiz 4: overloaded signatures

1. Can int f(int x) and double f(int x) appear together in the same class?

2. With f(int, double) and f(double, int), which overload handles f(2, 3.0)?

3. Why is f(2, 3) ambiguous?

Scope and lifetime of local variables

Scope determines where a name can be used.

A local variable's scope starts at its declaration and continues through its enclosing block.

A local variable needs a value before a read.
Each method call gets its own local variables.

A loop variable stays inside its loop

```
for (int i = 0; i < 3; i++) {
    int doubled = i * 2;
    System.out.println(doubled);
}
// System.out.println(i);       // error
// System.out.println(doubled); // error
```

Output: 0, 2, 4 on separate lines.

Separate blocks may reuse a name

```
for (int i = 1; i <= 3; i++) {
    System.out.println(i);
}
for (int i = 5; i <= 6; i++) {
    System.out.println(i);
}
```

The first i is out of scope before the second declaration.

A nested local cannot redeclare x

```
int x = 1;
if (x > 0) {
    int x = 0;  // compile-time error
    System.out.println(x);
}
```

Repair options: assign to the existing x, or use a new name.

Quiz 5: scope and assignment

1. Can two separate methods both declare x?

2. Can a for-loop variable be used after
   the loop when declared in its header?

3. Does this compile?
   ```
   int total;
   System.out.println(total);
   ```

Method abstraction and contracts

A caller needs the method's contract:

What inputs does it accept?
What result or action does it promise?
What assumptions must the caller satisfy?

The implementation can change while
the contract stays the same.

Stepwise refinement: a score report

Task: report the average and letter grade
for two valid scores.

Main task
1. Read each score
2. Compute the average
3. Convert the average to a grade
4. Display the report

A coordinating main method

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    double first = readScore(input);
    double second = readScore(input);
    double mean = average(first, second);
    char grade = letterGrade(mean);
    printReport(mean, grade);
}
```

Each method name describes one part of the report.

Score report: input and calculation

```
static double readScore(Scanner input) {
    System.out.print("Score (0 to 100): ");
    return input.nextDouble();
}

static double average(double a, double b) {
    return (a + b) / 2.0;
}
```

For scores 80 and 90, average returns 85.0.

Score report: grading and output

```
static char letterGrade(double score) {
    if (score >= 90) return 'A';
    if (score >= 80) return 'B';
    if (score >= 70) return 'C';
    if (score >= 60) return 'D';
    return 'F';
}

static void printReport(double mean, char grade) {
    System.out.printf("Average: %.1f%n", mean);
    System.out.println("Grade: " + grade);
}
```

For 80 and 90, the report shows Average: 85.0 and Grade: B.

Discussion: a change in requirements

The report must now accept three scores
and show the highest score.

Which methods can stay unchanged?
Which headers or calls need to change?
Where should input validation belong?

Exit quiz: designing a method

Design a method that counts multiples of 3 in an inclusive integer interval.

Write its header and describe its contract.
Then predict the result for (1, 10).
Name one boundary test.

Exit quiz: one possible solution

```
static int countMultiplesOfThree(int start,
                                 int end) {
    int count = 0;
    for (int n = start; n <= end; n++) {
        if (n % 3 == 0) count++;
    }
    return count;
}
```

Check: (1, 10) gives 3. (3, 3) gives 1. (4, 4) gives 0.

Chapter review

A method packages one coherent operation.
Parameters receive copied argument values.
A return value lets the caller use a result.
Signatures distinguish overloads.
Scope controls where a name is available.
Contracts support reuse and testing.